

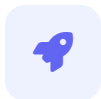
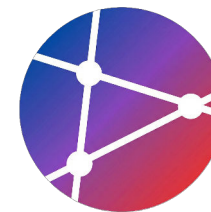
Deploying to Any Platform

다양한 플랫폼으로 배포하기

GitHub Pages, Vercel, Docker, AWS, GCP 등 주요 배포 환경 완전 정복

학습 목표

이 세션을 마치면 다음을 수행할 수 있습니다



자동 배포

다양한 플랫폼(Web, Serverless)에 대한 자동화된 배포 파이프라인 구축



컨테이너 배포

Docker 및 Kubernetes를 활용한 컨테이너 기반의 일관된 배포 환경 구현



클라우드 자동화

AWS, GCP 등 주요 클라우드 플랫폼에 대한 인프라 및 배포 자동화



배포 전략 수립

환경별(Dev, Staging, Prod) 특성에 맞는 최적의 배포 전략 및 프로세스 설계

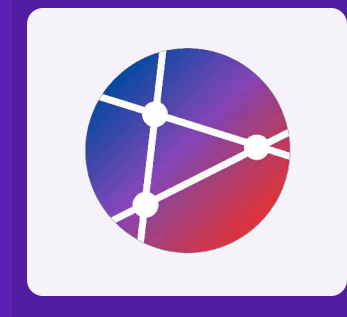


무중단 배포

Rolling Update, Blue-Green 등을 활용한 서비스 중단 없는 배포 구현

배포 플랫폼 개요

다양한 배포 환경(Static, PaaS, Container, IaaS)의
특징과 장단점을 비교하고, 프로젝트에 적합한 플랫폼
을 선택하는 기준을 알아봅니다.

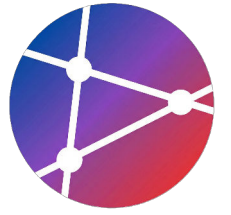


모
고
고

01

배포 플랫폼 분류

서비스의 규모와 요구사항에 따른 4가지 주요 배포 유형



Static Site Hosting

- ✓ GitHub Pages
- ✓ Vercel
- ✓ Netlify
- ✓ Cloudflare Pages



Platform as a Service

- ✓ Heroku
- ✓ Render
- ✓ Railway
- ✓ Fly.io



Container Platforms

- ✓ Docker
- ✓ AWS ECS/EKS
- ✓ Kubernetes
- ✓ Google Cloud Run

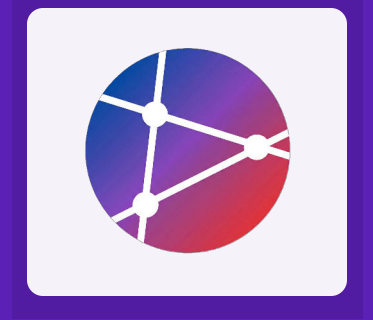


Cloud Providers (IaaS)

- ✓ AWS (EC2, S3)
- ✓ Microsoft Azure
- ✓ GCP
- ✓ DigitalOcean

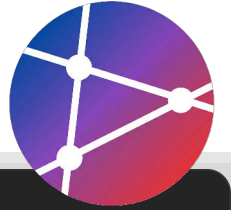
GitHub Pages 배포

정적 사이트 호스팅을 위한 무료 서비스인 GitHub Pages의 설정 방법과 GitHub Actions를 활용한 자동 배포 파이프라인 구축 방법을 다룹니다.



GitHub Pages 배포 (정적 사이트)

GitHub Actions를 활용한 정적 웹사이트 자동 배포 워크플로우 설정



.github/workflows/deploy-pages.yml

```
name: Deploy to GitHub Pages

on:
  push:
    branches: [main]      # main 브랜치 푸시 시 실행
  workflow_dispatch:      # 수동 실행 가능

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
```

- ✓ **Trigger:** main 브랜치에 코드가 푸시되거나 수동으로 트리거될 때 워크플로우가 시작됩니다.
- ✓ **Build Job:** Node.js 환경을 설정하고 의존성을 설치한 후 프로젝트를 빌드합니다.
- ✓ **Deploy Job:** 빌드된 아티팩트(dist 폴더)를 GitHub Pages 환경으로 배포합니다.

React / Vue / Angular 배포

프레임워크별 빌드 설정 및 환경 변수 구성 예시



.github/workflows/deploy-steps.yml

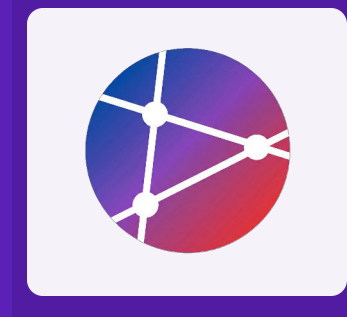
```
# 1. React Configuration
- name: Build React App
  run: |
    npm ci
    npm run build
  env:
    PUBLIC_URL: /repo-name # GitHub Pages 리포지토리 이름으로 경로 설정

# 2. Vue Configuration
- name: Build Vue App
  run: |
    npm ci
```

- ⚙️ **React:** PUBLIC_URL 환경 변수를 설정하여 정적 자산(CSS, JS)의 경로 문제를 해결합니다.
- ✔️ **Vue:** NODE_ENV: production 설정을 통해 개발 도구 경고를 제거하고 성능을 최적화합니다.
- ⚠️ **Angular:** --base-href 플래그를 사용하여 서브 디렉토리 배포 시 라우팅 오류를 방지합니다.

Vercel 배포

Next.js 개발사가 제공하는 최적화된 배포 플랫폼인 Vercel을 활용하여, 설정 파일 구성부터 GitHub Actions 연동, CLI 기반의 배포 자동화 과정을 다룹니다.



Vercel 배포 설정 (vercel.json)

프로젝트 배포 동작을 제어하기 위한 구성 파일 설정



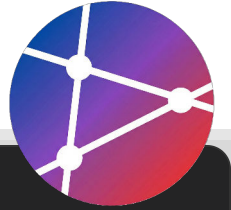
vercel.json

```
{
  "version": 2,
  "builds": [
    {
      "src": "package.json",
      "use": "@vercel/node"
    }
  ],
  "routes": [
```

- 🔗 **builds:** 어떤 소스 파일을 어떤 빌더(Builder)를 사용하여 컴파일할지 정의합니다. (예: Node.js, Static)
- 🔗 **routes:** URL 경로 재작성 및 리다이렉트 규칙을 설정합니다. SPA(Single Page Application)의 경우 모든 요청을 루트('/')로 보냅니다.
- 🔗 **env:** 배포 환경에서 사용할 환경 변수를 설정합니다. 주로 프로덕션 모드 설정 등에 사용됩니다.

GitHub Actions 통합 (Vercel)

Vercel Action을 사용하여 CI/CD 파이프라인 구축하기



.github/workflows/deploy-vercel.yml

```
name: Deploy to Vercel

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

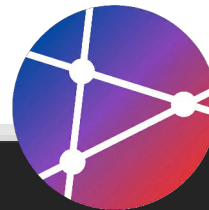
jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v3
```

- 🔑 **Required Secrets:** Vercel 계정 설정에서 Token, Org ID, Project ID를 발급받아 GitHub Secrets에 등록해야 합니다.
- 🔗 **Deployment Action:** amondnet/vercel-action을 사용하면 Vercel CLI 명령어를 래핑하여 쉽게 배포할 수 있습니다.
- 🔗 **Production vs Preview:** --prod 인자를 사용하면 프로덕션 환경에 배포하며, 생략 시 프리뷰 배포가 진행됩니다.

Vercel CLI 사용

터미널 명령어를 통한 프로젝트 배포 및 환경 변수 관리



Terminal / Bash

```
# 1. Vercel CLI 전역 설치
npm i -g vercel

# 2. Vercel 계정 로그인 (브라우저 인증 연동)
vercel login

# 3. 프로덕션 환경으로 배포 (Preview가 아닌 실제 운영 배포)
vercel --prod

# 4. 환경 변수 설정 (Production 환경에 Secret 추가)
# 형식: vercel env add [변수명] [환경]
vercel env add PRODUCTION_SECRET production

# 5. 프로젝트 연결 상태 확인 및 정보 조회
vercel project ls
```

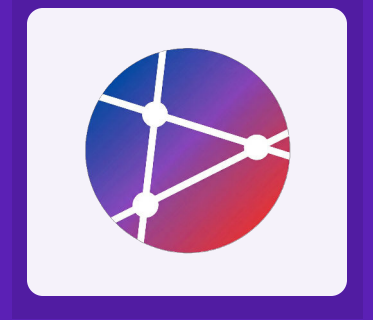
> **CLI Login:** `vercel login` 명령어로 로컬 개발 환경과 Vercel 계정을 안전하게 연동합니다.

🔑 **Production Deploy:** `--prod` 플래그를 사용하여 테스트 URL이 아닌 실제 도메인에 즉시 배포합니다.

🔑 **Environment Variables:** 웹 대시보드 접속 없이 CLI에서 직접 보안 키나 환경 변수를 관리할 수 있습니다.

Netlify 배포

정적 사이트 호스팅과 Serverless Functions를 위한
올인원 플랫폼인 Netlify의 설정 및 배포 자동화 과정을
다룹니다.



GitHub Actions 통합 (Netlify)

Netlify 자동 배포를 위한 CI/CD 워크플로우 설정



.github/workflows/deploy-netlify.yml

```
name: Deploy to Netlify

on:
  push:
    branches: [main]      # main 브랜치 푸시 시 배포
  pull_request:
    branches: [main]      # PR 생성 시 프리뷰 배포

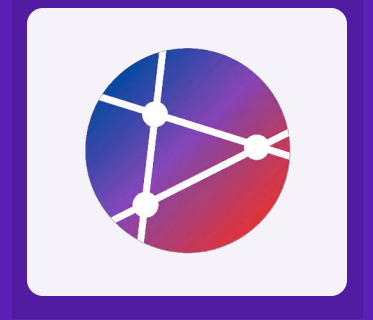
jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
```

- ✓ **Trigger:** main 브랜치 푸시(프로덕션 배포) 및 PR(프리뷰 배포) 시 워크플로우가 실행됩니다.
- 🔑 **Secrets:** Netlify 계정 설정에서 발급받은 AUTH_TOKEN과 SITE_ID를 GitHub Secrets에 등록해야 합니다.
- 🔧 **Action:** nwtgck/actions-netlify 액션을 사용하여 빌드된 결과물(dist)을 자동으로 배포합니다.

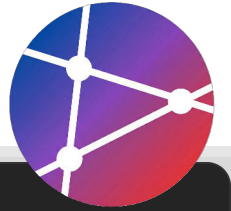
Docker 배포

컨테이너 기술의 표준인 Docker를 활용한 애플리케이션 패키징과 배포 프로세스를 학습하고, Dockerfile 및 Compose 설정 방법을 다룹니다.



Dockerfile 작성 (Multi-stage Build)

프로덕션 환경을 위한 최적화된 이미지 빌드 설정



Dockerfile

```
# 1. Builder Stage: 빌드 도구 포함
FROM node:18-alpine AS builder
WORKDIR /app

# 패키지 설치 (프로덕션용)
COPY package*.json ./
RUN npm ci --only=production

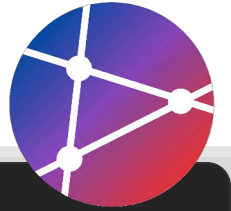
# 소스 코드 복사 및 빌드
COPY . .
RUN npm run build

# 2. Production Stage: 실행에 필요한 최소 환경
FROM node:18-alpine
```

- 📦 **Multi-stage Build:** 빌드 환경(Builder)과 실행 환경(Production)을 분리하여 최종 이미지 크기를 대폭 줄입니다.
- 🛡️ **Security:** USER node 지시어를 통해 컨테이너를 루트 권한이 아닌 일반 사용자 권한으로 실행하여 보안을 강화합니다.
- 📁 **Optimization:** COPY --from=builder를 사용해 실행에 꼭 필요한 빌드 결과물(`dist`)과 의존성(`node\_modules`)만 선택적으로 복사합니다.

Docker Compose 설정

멀티 컨테이너 애플리케이션 정의 및 실행을 위한 구성 파일



```
docker-compose.yml

version: '3.8'

services:
  app:
    build:
      context: .
      dockerfile: Dockerfile
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=${DATABASE_URL}
    depends_on:
      - db
  redis
```

 **Services:** Node.js 앱(app), PostgreSQL(db), Redis 캐시(redis) 3개의 독립된 컨테이너 서비스 정의

 **Networking:** `depends\_on` 옵션을 통해 서비스 실행 순서 제어 및 내부 네트워크 자동 연결

 **Volumes:** `postgres-data` 볼륨을 마운트하여 컨테이너가 재시작되어도 DB 데이터 영구 보존

GitHub Actions로 Docker 배포

이미지 빌드, 레지스트리 푸시 및 원격 서버 배포 자동화



.github/workflows/deploy-docker.yml

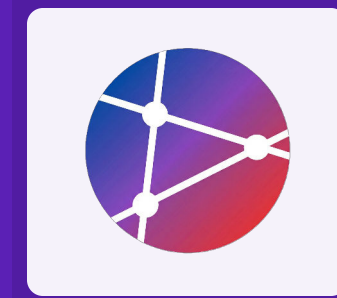
```
name: Deploy Docker
on:
  push:
    branches: [main]
    tags: ['v*']
env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${GITHUB_REPOSITORY}

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    permissions:
```

- 🔑 **Registry Login:** docker/login-action을 사용하여 GitHub Container Registry(ghcr.io)에 인증합니다.
- ⬆️ **Build & Push:** docker/build-push-action으로 이미지를 빌드하고 레지스트리에 자동으로 푸시합니다.
- 🖥️ **Remote Deploy:** appleboy/ssh-action을 통해 원격 서버에 접속하여 최신 이미지를 pull하고 컨테이너를 재시작합니다.

AWS 클라우드 배포

S3와 CloudFront를 이용한 정적 사이트 배포부터
Lambda Serverless, ECS 컨테이너 배포까지 AWS의
주요 배포 전략을 다룹니다.



AWS S3 + CloudFront 배포 (정적 사이트)

S3 버킷 동기화 및 CloudFront 캐시 무효화 자동화 워크플로우



.github/workflows/deploy-aws-s3.yml

```
name: Deploy to AWS S3

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-node@v3
        with:
```

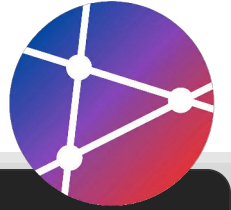
🔑 **AWS Credentials:** configure-aws-credentials 액션으로 GitHub Secrets에 저장된 키를 사용해 안전하게 인증합니다.

☁️ **S3 Sync:** aws s3 sync 명령어를 사용하여 로컬 dist 폴더와 S3 버킷의 내용을 동기화합니다.

🔄 **Invalidation:** 배포 직후 cloudfront create-invalidation을 실행하여 엣지 서버의 캐시를 갱신합니다.

AWS Lambda 배포 (Serverless)

GitHub Actions를 이용한 서버리스 함수 패키징 및 업데이트



.github/workflows/deploy-lambda.yml

```
name: Deploy to AWS Lambda

on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
```

- ✓ **Package:** npm ci --production으로 실행에 필요한 모듈만 설치 후 zip 명령어로 압축합니다.
- ✓ **Credentials:** configure-aws-credentials 액션으로 AWS CLI 인증 정보를 설정합니다.
- ✓ **Update:** aws lambda update-function-code 명령어로 기존 함수 코드를 갱신합니다.

AWS ECS 배포 (Container)

GitHub Actions를 활용한 AWS Elastic Container Service 자동 배포



.github/workflows/deploy-ecs.yml

```
name: Deploy to AWS ECS
on:
  push:
    branches: [main]

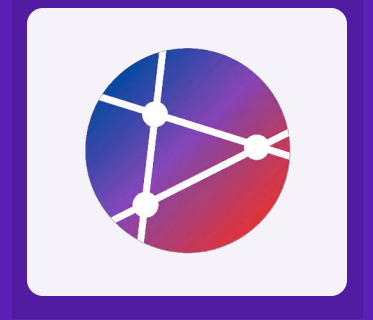
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3

      - name: Configure AWS credentials
        uses: aws-actions/configure-aws-credentials@v2
```

- 🔑 **AWS Credentials:** configure-aws-credentials 액션으로 AWS 권한을 설정하고 ECR에 로그인합니다.
- 🚢 **Build & Push:** Docker 이미지를 빌드하고 ECR 레지스트리에 푸시합니다 (Commit SHA 태그 사용).
- 🔧 **ECS Deployment:** 새 이미지를 기반으로 Task Definition을 업데이트하고 서비스를 재배포합니다.

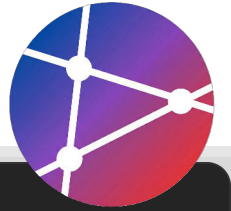
Google Cloud Platform 배포

Cloud Run을 활용한 서버리스 컨테이너 배포와
GitHub Actions를 연동한 완전 자동화된 CI/CD 파이프라인
구축 방법을 학습합니다.



Cloud Run 배포 (Serverless Container)

Google Cloud Run에 Docker 컨테이너를 자동으로 배포하는 워크플로우



.github/workflows/deploy-cloud-run.yml

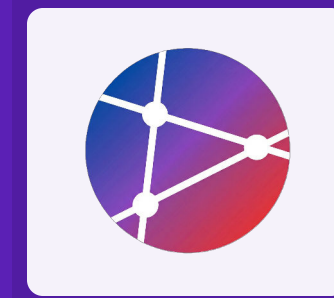
```
name: Deploy to Cloud Run
on:
  push:
    branches: [main]
env:
  PROJECT_ID: ${ secrets.GCP_PROJECT_ID }
  SERVICE_NAME: my-app
  REGION: us-central1

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
```

- ✓ **GCP 인증:** setup-gcloud 액션을 사용하여 서비스 계정 키(SA Key)로 인증합니다.
- ✓ **Build & Push:** Docker 이미지를 빌드하고 Google Container Registry(GCR)에 푸시합니다.
- ✓ **Cloud Run Deploy:** 푸시된 이미지를 기반으로 서버리스 컨테이너 서비스를 배포합니다.

환경별 배포 전략

개발(Dev), 스테이징(Staging), 프로덕션(Prod) 등 환경별 특성에 맞춘 효율적인 배포 파이프라인 구성과 전략을 알아봅니다.



환경별 배포 전략 (Multi-Environment Setup)

GitHub Actions Matrix Strategy를 활용한 브랜치별 자동 배포 환경 구성



```
.github/workflows/deploy-multi-env.yml

name: Multi-Environment Deploy

on:
  push:
    branches: [develop, staging, main] # 3가지 브랜치 감지

jobs:
  deploy:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        include:
          - branch: develop
            environment: development
```

- 📖 **Matrix Strategy:** 단일 job 정의로 여러 환경(develop, staging, production)을 효율적으로 관리합니다. 코드 중복을 최소화합니다.
- 🔗 **Conditional Logic:** `if: github.ref == ...` 조건을 사용하여 현재 푸시된 브랜치에 해당하는 매트릭스 설정만 실행되도록 제어합니다.
- 🔑 **Dynamic Secrets:** `secrets[format('{0}_API_KEY', ...)]` 구문을 통해 환경 이름에 따라 동적으로 시크릿 키를 참조하여 보안성을 높입니다.

Blue-Green Deployment (무중단 배포)

라이브 환경(Blue)과 대기 환경(Green)을 교체하여 다운타임 없이 배포하는 전략



.github/workflows/deploy-blue-green.yml

```
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy to Green
        run: |
          # 1. 새 버전을 격리된 Green 환경에 배포
          # 현재 사용자 트래픽은 여전히 Blue 환경으로 유지됨
          ./scripts/deploy_to_green.sh

      - name: Health Check
        run: |
          # 2. Green 환경이 정상 동작하는지 내부적으로 검증
```

- 🏗️ **Dual Environment:** 현재 서비스 중인 **Blue** 환경과 새 버전이 배포될 **Green** 환경을 동시에 운영합니다.
- 🔍 **Safe Verification:** 실제 트래픽을 받기 전, Green 환경에서 충분한 테스트와 헬스 체크를 수행할 수 있습니다.
- 🔄 **Instant Switch & Rollback:** 트래픽 전환만으로 배포와 롤백이 이루어져 다운타임이 거의 없고(Zero-downtime), 장애 대응이 매우 빠릅니다.

Canary Deployment (점진적 배포)

사용자 트래픽을 단계적으로 신규 버전으로 전환하여 리스크를 최소화하는 전략



.github/workflows/canary-deploy.yml

```
jobs:
  deploy-canary:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy Canary (10%)
        run: deploy_canary.sh 10 # 초기 10% 트래픽만 신규 버전으로 라우팅

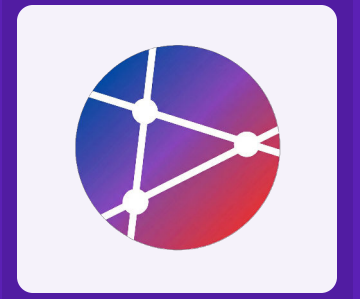
      - name: Monitor Phase 1
        run: sleep 300 # 5분간 로그 및 에러율 모니터링 (자동 검증)

      - name: Increase to 50%
        run: deploy_canary.sh 50 # 안정성 확인 후 트래픽 50%로 증대
```

- ✓ **Incremental Rollout:** 소수 사용자(10%)에게 먼저 배포하여 프로덕션 환경에서의 안정성을 검증합니다.
- ✓ **Automated Monitoring:** 각 배포 단계 사이에 대기 시간(Sleep)을 두어 시스템 상태를 자동으로 확인합니다.
- ✓ **Risk Mitigation:** 버그 발견 시 영향받는 사용자를 최소화하고 즉각적인 롤백이 가능합니다.

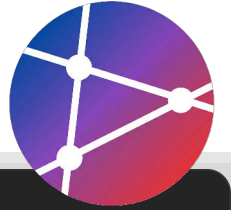
무중단 배포 (Zero-Downtime)

서비스 중단 없이 새로운 버전을 안정적으로 배포하는
Rolling Update, Blue-Green, Canary 전략과 자동화
된 롤백 메커니즘을 학습합니다.



Rolling Update (Kubernetes)

무중단 배포를 위한 Deployment 전략 및 Readiness Probe 설정



k8s-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
spec:
  replicas: 3
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1          # 새 파드를 최대 1개 더 생성 허용
      maxUnavailable: 0    # 배포 중 사용할 수 없는 파드 0개 (항상 100% 유지)
  template:
    spec:
```

- ✓ **RollingUpdate:** maxUnavailable: 0 설정으로 배포 중에도 항상 3개의 파드가 정상 작동하도록 보장합니다.
- ✓ **Readiness Probe:** 애플리케이션이 완전히 구동되고 `/health` 체크가 통과될 때까지 트래픽을 보내지 않습니다.
- ✓ **Zero Downtime:** 신규 버전이 준비된(Ready) 후에만 구버전을 종료하여 서비스 중단을 방지합니다.

Health Check & Rollback

배포 후 서비스 상태 검증 및 실패 시 자동 복구 로직



.github/workflows/deploy-check.yml

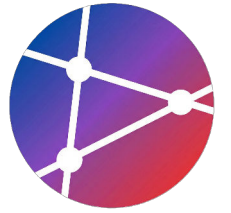
```
jobs:
  deploy-and-verify:
    runs-on: ubuntu-latest
    steps:
      - name: Deploy Application
        id: deploy
        run: |
          VERSION=$(date +%s)
          ./deploy.sh $VERSION
          echo "version=$VERSION" >> $GITHUB_OUTPUT

      - name: Health Check Loop
        run: |
```

- ▶ **Deploy:** 배포 스크립트를 실행하고, 롤백에 사용할 버전 ID를 Output 변수로 저장합니다.
- 💓 **Health Check:** curl을 사용하여 30회(5분간) 서비스 응답을 확인합니다. 실패 시 파이프라인을 중단합니다.
- ↺ **Auto Rollback:** if: failure() 조건으로 검증 실패 시 즉시 이전 버전으로 복구하는 스크립트를 실행합니다.

실습 과제

배포 역량 강화를 위한 단계별 실습 프로젝트



ASSIGNMENT 01

GitHub Pages 배포

React 또는 Vue 프로젝트를 생성하고 GitHub Actions를 사용하여 정적 웹사이트 호스팅 서비스인 GitHub Pages에 자동 배포 파이프라인을 구축합니다.

✓ CI/CD Pipeline



ASSIGNMENT 02

Vercel / Netlify 배포

Next.js 등 모던 프레임워크를 활용하여 PaaS 플랫폼에 배포합니다. 환경 변수 설정과 PR 생성 시 자동으로 생성되는 프리뷰 배포 기능을 구현합니다.

✓ Preview Deployment



ASSIGNMENT 03

Docker 컨테이너 배포

애플리케이션을 Docker 이미지로 빌드하고 Docker Compose를 통해 멀티 컨테이너 환경을 구성합니다. 자동화된 빌드 및 레지스트리 푸시를 구현합니다.

✓ Containerization



ASSIGNMENT 04

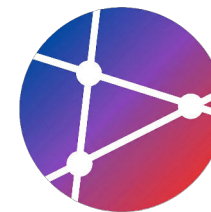
클라우드 플랫폼 배포

AWS 또는 GCP 계정을 사용하여 실제 클라우드 인프라에 서비스를 배포합니다. Serverless 또는 Container 서비스를 선택하여 운영 환경을 구축합니다.

✓ Cloud Infrastructure

과제 1: GitHub Pages 배포

React/Vue 프레임워크 기반의 정적 웹사이트 배포 실습



1

프로젝트 생성 및 설정

Create React App(CRA) 또는 Vue CLI를 사용하여 새로운 프로젝트를 생성하고, 로컬 환경에서 정상적으로 빌드되는지 확인하세요.

```
npx create-react-app
```

```
npm run build
```

2

GitHub Actions 워크플로우 작성

.github/workflows/deploy.yml 파일을 생성하고, main 브랜치 푸시 시 자동으로 빌드 및 배포되도록 설정하세요.

3

Custom Domain 설정 선택 사항 (Optional)

보유하고 있는 도메인이 있다면 CNAME 레코드를 설정하여 GitHub Pages에 커스텀 도메인을 연결해보세요.

제출 요구사항

✓ GitHub Repository URL

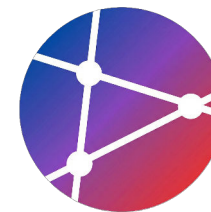
✓ 배포된 웹사이트 URL (Live)

✓ Actions 성공 로그 스크린샷

마감기한: 다음 주 수업 전까지

과제 2: Vercel/Netlify 배포

Next.js 또는 Nuxt 프레임워크 기반의 서버리스 배포 및 CI/CD 실습



1

프로젝트 생성 및 Git 연동

Next.js(React) 또는 Nuxt(Vue) 프로젝트를 생성하고 GitHub 리포지토리에 푸시하세요.

```
npx create-next-app
```

```
npx nuxi init
```

2

플랫폼 연결 및 환경 변수 설정

Vercel 또는 Netlify 대시보드에서 GitHub 리포지토리를 import하고, 프로젝트에 필요한 환경 변수(Environment Variables)를 설정하세요.

3

Preview & Production 배포 확인

PR 생성 시 자동으로 배포되는 Preview URL 기능과 Main 브랜치 병합 시 수행되는 Production 배포 파이프라인을 확인하세요.

```
Auto CI/CD
```

제출 요구사항



배포된 데모 사이트 URL



환경 변수 설정 화면 스크린샷

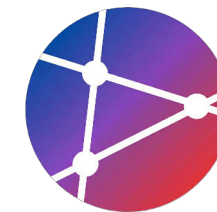


CI/CD 빌드 로그 스크린샷

마감기한: 다음 주 수업 전까지

과제 3: Docker 컨테이너 배포

컨테이너 기반의 애플리케이션 패키징 및 배포 파이프라인 구축



1

Dockerfile 작성

Multi-stage build를 활용하여 프로덕션용 경량 이미지를 생성하는 `Dockerfile` 을 작성하세요. Node.js Alpine 이미지를 베이스로 사용합니다.

2

Docker Compose 설정

App과 DB(PostgreSQL/MySQL) 서비스를 포함하는 `docker-compose.yml` 을 작성하고 서비스 간 네트워크 연결을 확인하세요.

3

CI/CD 파이프라인 구축

GitHub Actions를 사용하여 이미지를 빌드하고 Container Registry(GHCR 또는 Docker Hub)에 자동으로 Push하는 워크플로우를 구성합니다.

4

자동 배포 및 롤백

배포 스크립트를 통해 서버에서 최신 이미지를 Pull하여 실행하고, Health Check 실패 시 이전 버전으로 롤백하는 로직을 구현하세요.



제출 결과물



Docker Image Tag / URL



docker-compose.yml 파일

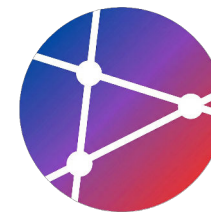


배포 및 롤백 테스트 로그

평가 기준: 이미지 크기 최적화 및 파이프라인 안정성

과제 4: 클라우드 배포

AWS/GCP 클라우드 서비스를 활용한 자동 배포 파이프라인 구축



1

클라우드 환경 준비

AWS 또는 GCP Free Tier 계정을 생성하고, 프로젝트에 적합한 컴퓨팅 서비스(Lambda, ECS, Cloud Run 등)를 선택하여 초기 설정을 완료하세요.

AWS Free Tier

GCP Free Tier

2

CI/CD 파이프라인 구축

GitHub Actions를 사용하여 클라우드 환경으로의 자동 배포 워크플로우를 작성하세요. IAM 사용자 권한과 Secret 키 관리에 주의해야 합니다.

3

모니터링 설정

필수 (Required)

배포된 서비스의 상태를 실시간으로 확인할 수 있도록 CloudWatch(AWS) 또는 Cloud Monitoring(GCP) 대시보드를 구성하세요.



제출 요구사항



배포된 서비스 URL (Live)



IaC 또는 Workflow 코드 파일

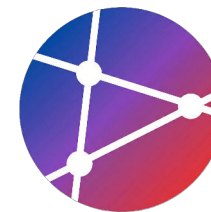


모니터링 대시보드 캡처

마감기한: 다음 주 수업 전까지

참고 자료 & 리소스

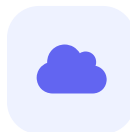
성공적인 배포 시스템 구축을 위한 공식 문서와 심화 학습 자료입니다.



필수 공식 문서

GitHub Pages, Vercel, Netlify 등 정적 호스팅 서비스와 Docker 컨테이너 플랫폼의 공식 가이드를 확인하세요.

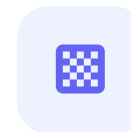
[View Documents →](#)



클라우드 플랫폼

AWS, Google Cloud Platform, Microsoft Azure 등 주요 퍼블릭 클라우드 서비스의 배포 및 관리 문서를 제공합니다.

[View Documents →](#)



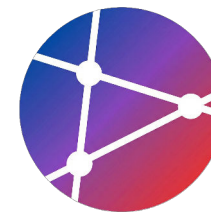
배포 전략 심화

무중단 배포를 위한 Blue-Green, Canary, Rolling Update 전략의 개념과 적용 사례를 학습할 수 있는 자료입니다.

[View Documents →](#)

참고 자료 (References)

강의 내용 심화 학습을 위한 필수 공식 문서 및 가이드



GitHub Pages Docs



GitHub 저장소를 통해 정적 웹사이트를 직접 호스팅하는 방법에 대한 공식 문서

<https://docs.github.com/pages>

Vercel Documentation



Next.js 및 다양한 프론트엔드 프레임워크의 배포와 서버리스 함수 설정 가이드

<https://vercel.com/docs>

Netlify Documentation



웹 개발 워크플로우 자동화 및 정적 사이트 호스팅을 위한 포괄적인 가이드

<https://docs.netlify.com/>

Docker Documentation

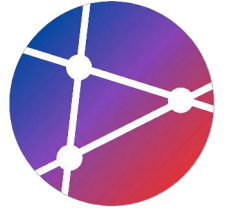


컨테이너화된 애플리케이션의 빌드, 공유, 실행을 위한 전체 매뉴얼

<https://docs.docker.com/>

참고 자료 - 클라우드 플랫폼

주요 클라우드 서비스 제공업체(IaaS/PaaS) 공식 문서 및 개발자 가이드



AWS Documentation

EC2, Lambda, S3 등 AWS 서비스의 상세 가이드 및 API 레퍼런스

<https://docs.aws.amazon.com/>



Google Cloud Docs

Google Cloud Platform의 Cloud Run, GKE 등 서비스별 공식 문서

<https://cloud.google.com/docs>



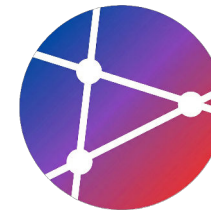
Microsoft Azure Documentation

Azure 클라우드 서비스, 아키텍처 센터, 빠른 시작 튜토리얼 및 개발자 리소스

<https://docs.microsoft.com/azure>

참고 자료 - 배포 전략

안정적이고 효율적인 무중단 배포를 위한 핵심 개념 가이드



Blue-Green Deployment

두 개의 동일한 프로덕션 환경을 교대로 사용하여 다운타임 없이 배포하고 즉시 롤백을 가능하게 하는 전략 (Martin Fowler)

<https://martinfowler.com/bliki/BlueGreenDeploym...>



Canary Releases

새 버전을 소수의 사용자에게 먼저 배포하여 전체 적용 전 위험을 감지하고 점진적으로 트래픽을 늘리는 배포 기법

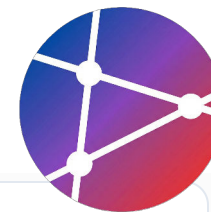
<https://martinfowler.com/bliki/CanaryRelease.html>

핵심 요약

기억해야 할
Key Takeaways



오늘 학습한 배포 플랫폼과 전략들을 실무에 적용하기
위해 반드시 기억해야 할 5가지 핵심 포인트입니다.



1

프로젝트 특성에 맞는 플랫폼별 배포 방식 이해하기

2

Dev, Staging, Prod 등 환경별 배포 전략 수립

3

서비스 중단을 방지하는 무중단 배포(Zero-Downtime) 구현

4

배포 실패 시 즉시 복구할 수 있는 롤백(Rollback) 계획 필수

5

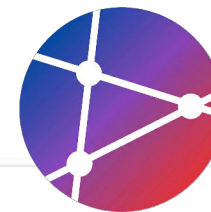
배포 후 안정성을 보장하는 자동화된 헬스 체크 설정

Best Practices

성공적인 배포를 위한
실무 가이드



안정적이고 효율적인 배포 파이프라인 구축을 위해 권장
되는 6가지 핵심 실천 사항입니다.



환경별 분리: 개발(Dev), 스테이징(Staging), 운영(Prod) 환경의 철저한 격리



자동화된 헬스 체크: 배포 직후 서비스 상태를 자동으로 검증하여 장애 예방



롤백 메커니즘: 문제 발생 시 즉시 이전 안정 버전으로 복구하는 체계 마련



배포 모니터링: CI/CD 파이프라인 및 배포 후 애플리케이션 지표 실시간 추적



점진적 배포 (Canary): 일부 사용자에게 먼저 배포하여 전체 장애 리스크 최소화



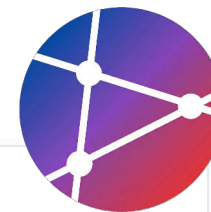
Infrastructure as Code (IaC): 인프라 설정을 코드로 관리하여 환경 일관성 유지

다음 단계

Action Plan
For Deployment



오늘 학습한 내용을 바탕으로 실제 프로젝트에 적용하기
위한 구체적인 실행 계획입니다.



1

프로젝트 요구사항에 맞는 배포 플랫폼 선택 및 구축

2

GitHub Actions를 활용한 CI/CD 파이프라인 완성

3

실시간 장애 감지를 위한 모니터링 및 알림 설정

4

긴급 상황을 대비한 재해 복구(Disaster Recovery) 계획 수립

“

"Deploy early, deploy often."

자주 배포하고, 빠르게 피드백을 받아 개선하세요!